MobiShard: Lifecycle-Aware Resource Reservation for Mobile Smart-Contract Validation

Vipin Deval*†, Vimal Dwivedi‡, Alex Norta§¶, Dirk Draheim†
*Department of Computer Science and Engineering,
Krishna Institute of Engineering & Technology (KIET), Ghaziabad, India
†Information Systems Group, Tallinn University of Technology, Tallinn, Estonia
‡Intelligent System Research Center, Center for Legal Technology,
Ulster University, United Kingdom
§Tallinn University (TLU), Tallinn, Estonia ¶University of Pretoria (UP), Pretoria, South Africa
Corresponding author: Vipin Deval (vipin.deval@taltech.ee)

***Abstract*—Mobile smart-contract requests can pass admission checks yet remain unfinished when concurrent requests draw on the same budget. Checking each validation stage prevents negative balances, but does not protect the allowance needed by later stages. We present MobiShard, which atomically reserves each request's remaining logical battery and computation costs before shard dispatch. Stage charges reduce the reservation, and terminal handling releases any unused portion. The design separates request origination, the charged account, and execution placement. A budget invariant establishes conditional resource feasibility under fixed costs and atomic updates. Selected guards and evidence dependencies come from a Coloured Petri Net specification and inform a separate implementation. We compare stagewise checking, full-path prechecking, reservation, and per-participant serialization in a trusted, single-host prototype with six services. The evaluation covers 880 trials and 225,280 submissions. In computation-limited bursts, reservation completes 80 of 256 requests per trial, compared with a mean of 72.40 for stagewise checking, a 10.5% increase. Serialization also reaches 80, but permits only one active request per participant. In mixed-resource trials, both complete 96 requests; reservation reduces mean trial elapsed time from 377.2 to 220.6 ms. No clear timing advantage is established in the main computation-limited comparison. Modeled connectivity loss, however, interrupts more admitted requests under reservation than under serialization. Reservation thus protects completion capacity while allowing concurrent validation, but its runtime benefit depends on the workload. These results concern logical budgets and local commitment, not measured mobile-energy savings or consensus finality.**

***Index Terms*—mobile smart contracts, resource reservation, admission control, sharding, Coloured Petri Nets, concurrent validation**

I. Introduction

Edge assistance lets a mobile participant submit smart-contract requests and inspect validation evidence without running the entire blockchain on its device. Mobile smart-contract (MSC) participation still requires a sequence of resource-consuming operations, which may run on different hosts and at different times. Saguaro uses a hierarchy to process mobile-originated transactions [1]; permissioned sharding and cross-shard protocols also divide processing among services [2]–[4]. A balance read at admission therefore describes only part of the account's position. The account may have enough allowance for the next operation while owing resources to requests already in progress. We study those outstanding obligations using logical charges to a participant's account, rather than measurements of its device's physical resource consumption.

Consider several service-payment requests submitted by a mobile participant during a short connectivity window. Each can pass the resource check for its next validation stage. If they all spend on early stages before any finishes, the account may be left with too little allowance for the remaining work. No stage has overdrawn the account, yet the requests can collectively complete fewer paths than the initial budgets could support. An admission check for the full path has the same weakness when it records no commitment: until consumption occurs, later requests can be checked against the same balance. This is the difference between *observing available resources* and *reserving the resources required to complete an admitted request*. Throughout the paper, completion capacity means the number of requests that the initial wallet and resource budgets can support through the configured validation path.

Where a request executes and how its allowance is committed are separate decisions. TxAllo optimizes transaction allocation [5], and BrokerChain addresses account/balance-based cross-shard processing [6]. Placement and coordination mechanisms do not, on their own, determine how to share a participant's remaining allowance among unfinished requests. Resource management across an entire processing path is already an established concern in multi-stage services [7]. Here, we examine its interaction with stage-level eligibility checks, evidence-dependent acceptance, and concurrent validation. Reservation must therefore be compared not only with checks that record no hold, but also with the simpler option of processing each participant's requests one at a time.

The research question is: *Under what conditions does reserving the remaining validation cost before shard dispatch preserve completion capacity, and what runtime trade-offs arise relative to checks alone and per-participant serialization?* MobiShard makes the commitment at admission. Before a request is dispatched, an atomic operation reserves its remaining validation allowance; subsequent stage charges consume the reservation, and terminal handling releases any unused part. The path starts with deposit-backed request creation, uses

deterministic routing and candidate construction, and passes symbolic evidence checks before reaching a local commitment gate. A Coloured Petri Net (CPN) specification supplies the design-time types, guards, and cost rules. The experiments run a separate service implementation of selected rules, with trusted services and a local commitment record as the completion event. That event does not represent blockchain consensus finality.

The architecture identifies who submits a request, which account pays its logical cost, and which service executes each operation. Reservation updates and terminal feedback are explicit. We formulate an atomic admission rule and an accounting invariant: with fixed costs and the specified updates, another admitted request cannot spend a reserved allowance. The implementation then compares stagewise checking, full-path prechecking without reservation, lifecycle reservation, and per-participant serialization. Resource profiles, concurrency, arrivals, instrumentation, and connectivity are varied to separate avoiding partial-work losses from the runtime benefit of overlap. These form the paper's architectural, accounting, and experimental contributions.

Under the successful-path assumptions, both reservation and serialization reach the configured static resource bound. Reservation completes 80 requests per 256-request computation-limited burst trial, against a mean of 72.40 for stagewise checking; serialization also completes 80. Reservation thus avoids partial-work losses while allowing overlapping execution, but does not consistently improve latency over serialization. Nor does its accounting guarantee ensure progress after connectivity is lost. Section II reviews the related work. Section III describes the architecture and its assumptions, Section IV presents the experiments and their limitations, and Section V concludes the paper.

II. Related Work

Three strands of work bear on this admission problem: sharded transaction processing, mobile and edge participation, and resource management in multi-stage services. We review each in relation to the budget that unfinished validation requests still need, distinguishing that question from the other objectives these systems pursue.

A. Sharding, Placement, and Transaction Coordination

A sharded system must distribute processing and coordinate operations that cross partition boundaries. SharPer replicates shards within network clusters and coordinates transactions across those clusters [2]. ByShard studies atomic commitment and isolation in a Byzantine environment [3], while Rivet places consensus in a reference shard and transaction processing in worker shards [4]. Although their arrangements differ, each separates aspects of execution from agreement. The guarantees obtained depend on the coordination protocol, not merely on assigning work to different services.

Placement and data exchange determine part of the overhead within these systems. TxAllo allocates transactions using their relationships [5]; SPRING applies deep reinforcement learning to state placement [8]. BrokerChain supports account/balance-based cross-shard processing through broker accounts [6]. For sharded databases, GriDB moves cross-shard data exchange off-chain and uses consensus to verify the resulting evidence [9]. These are not the variables changed in MobiShard's policy comparison. Its routing map remains fixed, so differences in admission and completion can be examined without simultaneously changing placement or coordination.

Shard performance also depends on node capabilities and committee duties. RepChain represents validator heterogeneity through reputation and uses it to support cooperation [10]. ContribChain combines contribution-aware node and account allocation to match workload with shard processing capability [11]. StableShard separates proposer shards from finalizer committees [12]; SpiralShard secures smaller shards through linked cross-shard endorsement [13]. These choices determine who processes a request and how shard results become trustworthy. MobiShard takes a trusted service configuration and an identified resource account as given, then asks how concurrent requests should share that account's remaining allowance.

Contention over a budget is not the same problem as conflicting access to transaction state. Prophet avoids cross-shard transaction conflicts through Byzantine-tolerant deterministic ordering [14]. Lockless sharding uses multiversion control [15], and Block-STM allows parallel execution while preserving the result of a prescribed transaction order [16]. MobiShard reserves a logical validation allowance; it does not enforce transaction isolation. The term serialization also has a limited meaning in its baseline: requests charged to the same participant cannot overlap. That rule is not a database-serializability guarantee, and the local commitment record is not the consensus finality of a complete blockchain protocol.

B. Mobile Participation, Edge Resources, and Lightweight Evidence

In mobile and edge settings, resource allocation is closely tied to where computation runs and which participants can support it. Saguaro places mobile-originated transactions in a hierarchy of edge and higher-level services [1]. Qin et al. jointly optimize wireless, computation, and consensus-related energy use in user-centric mobile edge computing [17]. Wang et al. divide edge-server resources between offloaded tasks and blockchain-based federated learning, subject to cost and completion-time constraints [18]. TBDD considers trust evaluation and adaptive node allocation for secure IoT participation [19]. Together, these studies address physical resource allocation and participant selection. The constraint in MobiShard is instead a logical allowance that does not replenish during a trial. The account receives the charge regardless of which host executes the stage.

Light-client protocols address the burden of obtaining and checking blockchain evidence. FlyClient uses compact proofs for lightweight chain and transaction verification [20]; Plumo supports efficient synchronization through succinct state-transition proofs [21]. Both concern the evidence a client must receive and validate. In MobiShard, symbolic predicates

over candidate metadata and deposit records represent the dependencies required for acceptance. This abstraction keeps the experiment focused on admission and does not implement the light clients' cryptographic verification mechanisms.

C. Admission, End-to-End Resource Accounting, and Formal Models

Managing a request end to end requires looking beyond its next processing stage. InferLine provisions prediction pipelines to meet end-to-end latency objectives [7]. Clockwork uses predictable execution times to meet request-level performance objectives [22], and Sinan accounts for dependencies between microservices when allocating resources [23]. Caladan deals with host-level interference through rapid CPU-core allocation [24]. These systems adjust serving capacity or execution schedules. MobiShard makes a narrower accounting decision: how much of an account's remaining budget to commit to a request's unfinished stages at admission. Reserving that budget does not assign a physical core or guarantee a response-time deadline.

Resource dimensions also matter in blockchain fee mechanisms. Dynamic multidimensional pricing assigns prices to different scarce resources [25], while Angeris et al. analyze the welfare properties of the price updates [26]. Lavee et al. ask how much capacity multidimensional constraints add relative to their complexity [27]. Pricing resources and determining feasible block capacity are related to, but distinct from, holding an individual account's remaining lifecycle allowance. The costs used by MobiShard are configured logical quantities rather than calibrated gas units. Its deposit is debited separately at admission.

A formal model can expose the stage dependencies and accounting conditions that an implementation must retain. CPN IDE provides an extensible CPN modeling environment [28]; SimPN offers timed, colored Petri-net simulation in Python [29]; and CPN-Py supports modeling and state-space analysis [30]. Their role here is methodological, not that of competing admission policies. Selected types, guards, and cost rules from the CPN guide the separate service implementation. Replay checks recorded executions against the scoped accounting and ordering conditions. It neither establishes full behavioral equivalence with the CPN nor exhaustively explores its reachable states.

The comparison in this paper concerns how admission policies account for unfinished work. MobiShard combines early reservation, stage-level consumption, and release of unused holds with the existing downstream evidence checks. Stagewise checking, a full-path precheck without reservation, lifecycle reservation, and per-participant serialization are evaluated with the same routing configuration and stage rules. This isolates when reservation avoids partial-work losses and whether the overlap it permits offers a useful runtime trade-off against serialization. Table I places that question alongside the objectives reviewed above and provides the basis for the architecture that follows.

TABLE I
OBJECTIVES OF REPRESENTATIVE RESEARCH DIRECTIONS AND THEIR RELATIONSHIP TO MOBISHARD. THE COMPARISON DISTINGUISHES SCOPES RATHER THAN RANKING FEATURE COMPLETENESS.

Research direction	Representative objective	Relation to MobiShard
Sharding and placement	Distribute state and coordinate processing [2], [5], [9].	Routing is fixed to isolate account-level admission decisions.
Concurrency control	Preserve consistent state access during parallel execution [14], [16].	Reservation protects validation budgets, not transaction isolation.
Mobile and edge resources	Allocate computation and communication under service constraints [17], [18].	The allowance is logical and separate from execution placement.
Lightweight verification	Reduce client-side evidence and verification costs [20], [21].	Symbolic evidence gates delimit the admission experiment.
End-to-end resource management	Provision or schedule services against performance objectives [7], [22], [23].	The comparison measures completion and runtime against serialization.
Multi-dimensional fees	Price resources and characterize feasible block capacity [25]–[27].	Per-request reservations are distinct from fees and deposit debits.
CPN tooling	Model, simulate, or analyze typed workflows [28]–[30].	Selected rules guide implementation and checks on recorded traces.

III. PROPOSED SYSTEM ARCHITECTURE

The architecture gives the gateway responsibility for the allowance promised to admitted work, while execution services remain responsible for their stage-level resource and evidence checks. This division implements the distinction between accounting, placement, and coordination made in the preceding sections. We first define the roles and assumptions, then follow a request through the services before establishing why its reservation can cover the remaining path.

A. Roles, Resource Ownership, and Assumptions

Request origination, resource ownership, and execution placement need not coincide. The *originator* submits the request; the *resource owner* is the account charged for validation; and the *execution host* performs the service operation. The experiments use the same identifier for originator and owner, but run the gateway, shard workers, and root service in separate processes. A root-validation charge to the owner's account therefore describes which account bears the logical cost, not a computation carried out on the owner's phone.

A request carries $x = (r, u, k)$, where r is its unique identifier, u is the registered owner, and k selects its routing key. Requests may share a routing key while retaining distinct identifiers. For owner u, the stored state comprises wallet balance W_u, remaining resources $\mathbf{a}_u = (B_u, C_u)$, and connectivity state n_u. Battery-budget and computation-budget units are accounting quantities: a stage charge reduces them, and the trial does not replenish them. They are neither battery percentages nor processor utilization readings. Connectivity is

represented in the same abstract way. Setting $n_u = 0$ makes the owner ineligible at a connectivity guard; the prototype's loopback services remain connected.

The argument depends on five assumptions. *A1: fixed accounting costs.* Every request uses the block, proof, and root stages in that order. Their costs are known and non-negative, and the account is not subject to unaccounted resource drain. *A2: one charged owner.* The same account bears all selected stage charges, even when different hosts perform the operations. *A3: atomic state updates.* The trusted gateway performs the admission check, deposit debit, and reservation update together. It also applies each stage charge together with the checks on mutable conditions. Request state prevents repeated charging of a stage and further execution after termination. *A4: scoped evidence.* Trusted services exchange well-formed symbolic records. Proof construction and root validation require the retained predicates to hold. *A5: conditional progress.* The connectivity guards must remain satisfied where required, the services must respond, and enabled operations must ultimately be scheduled. The later-disconnection experiment deliberately changes the connectivity condition.

Requests in the experiment consist of deposit and validation records. The implementation follows their resource-dependent validation path; it does not execute application contract logic, a virtual machine, committee consensus, or receipt-based multi-shard state updates. Service-crash handling and recovery from lost calls are not part of the failure model either. The experiment can therefore examine the accounting mechanism without claiming to implement a complete mobile blockchain.

B. Components and End-to-End Flow

In Fig. 1, the participant interface supplies the request identifier and owner binding. The gateway handles admission before execution services process the request. It requires a mapped routing key and enough wallet balance for the deposit. Reservation additionally checks connectivity and the balance left after outstanding holds are excluded. A successful decision records the hold and debits the deposit atomically. Otherwise the gateway denies the request without dispatch, deposit debit, or modeled validation charge.

The account table records each owner's balances and aggregate reservation. A separate request ledger records ownership, routing assignment, the remaining hold, the last charged stage, and terminal status. These records distinguish what remains in the account from what is available to newly arriving work: part of the balance may already be promised to unfinished requests. The stage marker tracks charges rather than completed processing. For example, the block-stage cost is entered before the configured block delay has elapsed.

Edge packaging precedes worker selection through the fixed routing map. A worker serializes its block-stage processing and calls the gateway for the stage check and charge. Candidate construction takes place in the gateway, which also updates the global and shard-specific identifiers and hash-related state. The worker waits for the block delay, checks the proof inputs, requests the proof-stage charge, and constructs a symbolic proof. It sends that proof and the candidate to the root service. Thus, the worker schedules block processing but does not itself construct the candidate. The four workers are processing services, not four blockchains with independent consensus.

At the root service, the candidate and proof identifiers must agree and the retained root-policy predicates must pass. The service then requests the root-stage charge and records a positive decision for that candidate. A local commitment can be recorded after the configured delay only with the candidate and its matching decision. The terminal outcome returns through the gateway along the request path. The three outcomes distinguish how far processing reached: *committed* means that the local record was made; *blocked* means that an admitted request stopped at a handled resource or evidence check; and *denied* means that the request never entered processing. Terminal handling releases the remaining hold. Neither earlier resource charges nor the admission deposit are refunded.

A reserved budget does not exempt a request from later checks. Modeled connectivity loss can make the owner ineligible at a subsequent guarded stage and stop work for which resources are still reserved. The check is stage-specific: after root acceptance, the local commitment timer does not test connectivity again. Similarly, a release depends on reaching terminal handling. The implementation does not automatically recover an unused hold from an unhandled service failure.

C. Why a Precheck Is Not a Reservation

Take an owner with 14 computation-budget units and enough battery and wallet allowance to proceed while connectivity remains stable. Each request needs 3 units for block generation, 2 for proof construction, and 2 for root validation. Before any charge, a full-path precheck can accept three requests because each is individually affordable at seven units. Now let all three execute their block stages, spending 9 units, and let two construct proofs, spending a further 4. The one unit left cannot pay for either root validation or the third request's proof. This ordering completes no requests, even though every executed stage passed its guard and the balance stayed non-negative.

Reservation makes a different admission decision: the first two requests reserve the combined 14-unit cost and the third is denied. The admitted requests can then complete under A1–A5. This example describes an allowed ordering, not a measured run or an unavoidable outcome of prechecking. A schedule that finishes one request before starting the next can also produce two commitments. Hence the experiment compares reservation with serialization as well as unreserved checks; both accounting and execution order can prevent spending on work that never finishes.

The policies differ in what admission does with the available allowance. *Stagewise checking* checks routing and the wallet first, leaving resource and connectivity checks to each charged stage. A *full-path precheck* additionally tests connectivity and the total path cost before admission, but leaves the balance unreserved. *Lifecycle reservation* uses the unreserved balance for that decision and retains a per-request hold throughout processing. *Per-participant serialization* queues an owner's

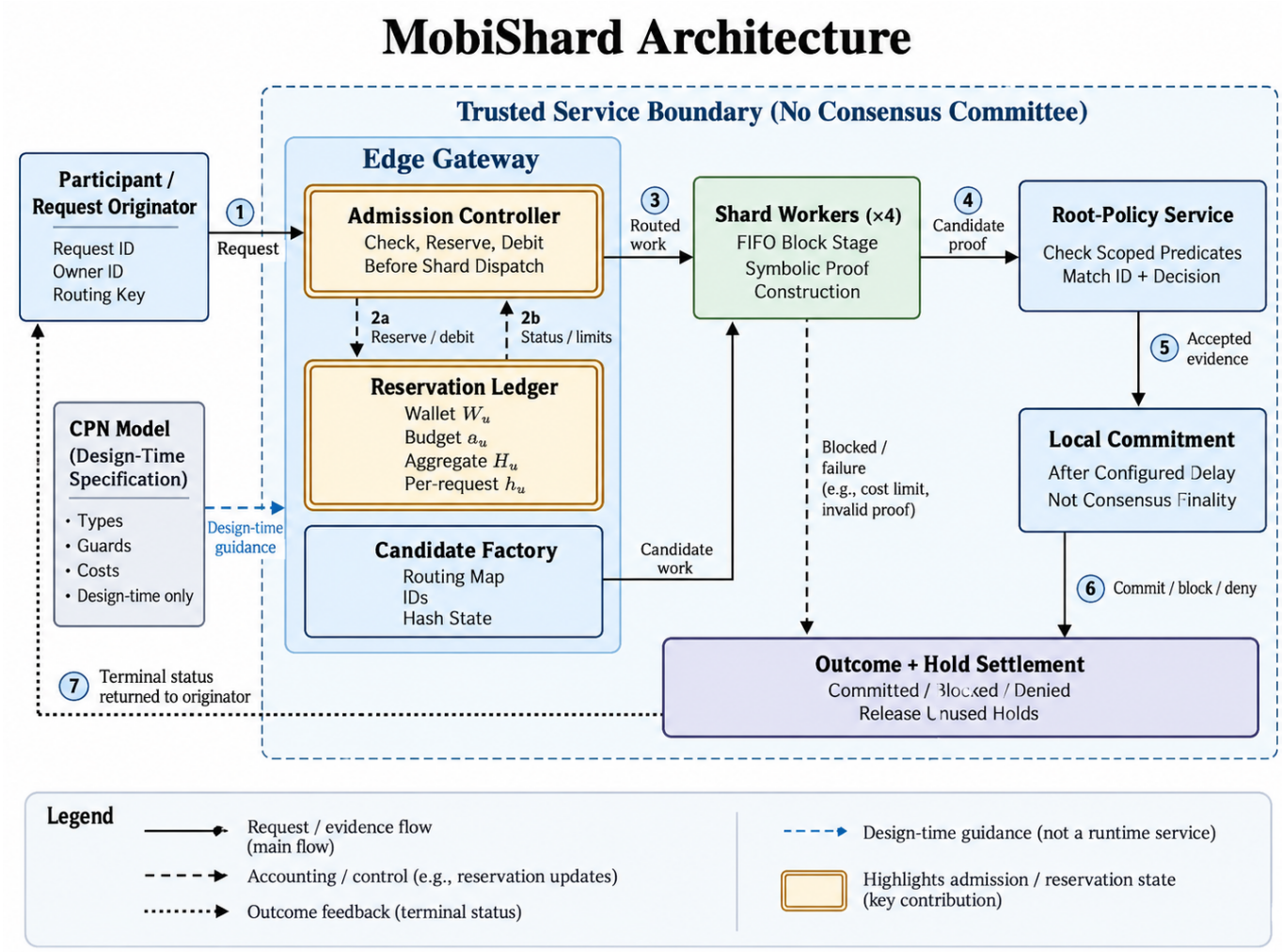


Fig. 1. MobiShard service allocation. The edge gateway maintains admission, reservation, and candidate-construction state. Four logical shard workers schedule block-stage calls and construct symbolic proofs; the root-policy service checks evidence and records local commitment. Terminal handling returns outcomes and releases unused reservations. The CPN supplies design-time rules rather than acting as a runtime service.

requests before the full-path precheck. It waits for the current request's terminal response before admitting the next, without preventing work from other owners. All four use the same routing, stage costs, and evidence conditions.

D. Reservation Rule and Accounting Invariant

The ordered sequence $J = (b, p, r)$ consists of block generation, proof construction, and root validation. Writing battery before computation, the configured stage costs and their total are

$$\begin{aligned} &\mathbf{q}_b = (2,3), \quad \mathbf{q}_p = (1,2), \quad \mathbf{q}_r = (1,2), \\ &\mathbf{Q} = \mathbf{q}_b + \mathbf{q}_p + \mathbf{q}_r = (4,7). \end{aligned} \tag{1}$$

For an active request x, J_x contains the remaining stages in order, excluding any that have already been charged. Its remaining reservation is $\mathbf{h}_x$. If $\mathcal{A}_u$ denotes the active requests charged to owner u, then the per-request and aggregate reservations are

$$\mathbf{h}_x = \sum_{j \in J_x} \mathbf{q}_j, \qquad \mathbf{H}_u = \sum_{x \in \mathcal{A}_u} \mathbf{h}_x.$$

The sum is zero when no stages remain. Denied and terminal requests retain no reservation, and vector comparisons throughout are componentwise.

For a new request from a registered owner, assume a unique request identifier and a positive deposit d. The gateway admits it under Reservation when

$$\begin{aligned} &\text{mapped}(k) \ \wedge \ W_u \geq d \ \wedge \ n_u \neq 0 \\ &\qquad \wedge \ \mathbf{a}_u - \mathbf{H}_u \succeq \mathbf{Q}. \end{aligned} \tag{2}$$

The gateway evaluates this condition and records the following changes in one atomic operation:

$$W_u \leftarrow W_u - d, \quad \mathbf{h}_x \leftarrow \mathbf{Q}, \quad \mathbf{H}_u \leftarrow \mathbf{H}_u + \mathbf{Q}. \tag{3}$$

The request joins $\mathcal{A}_u$ with its first stage pending. Admission changes how the allowance is assigned, not how much has been spent: $\mathbf{a}_u$ is unchanged.

Only an active request's next stage may be charged, after its input and evidence checks pass. The gateway evaluates the remaining mutable conditions and applies the charge atomically:

$$n_u \neq 0 \ \wedge \ \mathbf{a}_u \succeq \mathbf{q}_j \ \wedge \ \mathbf{h}_x \succeq \mathbf{q}_j. \tag{4}$$

A successful check permits the following stage charge:

$$\begin{aligned} &\mathbf{a}_u \leftarrow \mathbf{a}_u - \mathbf{q}_j, \\ &\mathbf{h}_x \leftarrow \mathbf{h}_x - \mathbf{q}_j, \qquad \mathbf{H}_u \leftarrow \mathbf{H}_u - \mathbf{q}_j. \end{aligned} \tag{5}$$

Charging and advancing the expected stage are one operation. The request spends its existing reservation; it does not repeat the admission test against the unreserved balance.

For a handled terminal outcome, the gateway releases whatever reservation remains:

$$\mathbf{H}_u \leftarrow \mathbf{H}_u - \mathbf{h}_x, \qquad \mathbf{h}_x \leftarrow \mathbf{0}. \tag{6}$$

Settlement also removes the request from $\mathcal{A}_u$ and records its terminal status in the same atomic operation. New work can use the released allowance, but neither $\mathbf{a}_u$ nor W_u increases. A path for which every stage has been charged already has $\mathbf{h}_x = \mathbf{0}$. That value records exhaustion of the hold, not acceptance of the candidate: matching evidence and the local timer are still required for commitment. Algorithm 1 collects the admission, charging, and settlement steps.

Proposition 1 (Accounted resource feasibility)**.** *Assume A1–A3, non-negative initial budgets, and initially empty reservations. If the state changes only through the guarded operations above, every update preserves*

$$\mathbf{a}_u \succeq \mathbf{H}_u \succeq \mathbf{0}. \tag{7}$$

The reservation of each active request also equals the cost of its uncharged stage suffix. Thus, work on another admitted request cannot consume the allowance assigned to those remaining configured stages.

Proof. The initial state has no active reservations, so $\mathbf{H}_u = \mathbf{0}$. For admission, the test on unreserved balance permits $\mathbf{H}_u$ to increase by $\mathbf{Q}$ and initializes $\mathbf{h}_x$ to the full path cost. A stage charge subtracts the same non-negative vector from $\mathbf{a}_u$, $\mathbf{h}_x$, and $\mathbf{H}_u$. The difference between account balance and aggregate hold is therefore preserved. Fixed costs and the next-stage check maintain the suffix relation, and the check on the request's hold prevents a negative reservation. Settlement subtracts the remaining hold without reducing the account balance. These operations preserve the aggregate invariant and the per-request relation by induction. Immediately before the next stage j, they give $\mathbf{a}_u \succeq \mathbf{H}_u \succeq \mathbf{h}_x \succeq \mathbf{q}_j$. The stage's accounted resources are consequently available regardless of other correctly charged requests. Evidence validity and continued service or network availability are separate requirements. □

Algorithm 1. Lifecycle reservation and settlement

Input: new request $x = (r, u, k)$ from registered owner u.
Atomically at the gateway: check Eq. (2);
  if false, return denied; otherwise apply Eq. (3)
  and initialize the active request's stage state.
Package the admitted request and route it to its worker.
for each stage j in block, proof, root order **do**
  Check the stage's input and evidence conditions.
  if a required condition fails **then**
    Settle as blocked using Eq. (6); **return**.
  end if
  Atomically at the gateway: require active status
    and the expected stage; test Eq. (4); if true,
    apply Eq. (5) and record the stage charge.
  if the stage charge was refused **then**
    Settle as blocked using Eq. (6); **return**.
  end if
  Complete stage processing and carry its evidence forward.
end for
Require the candidate's matching positive root decision.
Wait for the local commitment delay and record commitment.
Atomically settle using Eq. (6); return committed.

Let N_u count distinct, mapped requests offered by owner u, with *initial* balances W_u^0, B_u^0, and C_u^0. Each completion consumes one deposit and the full cost in Eq. (1). The static resource-capacity bound is

$$K_u = \min\left\{ N_u, \left\lfloor \frac{W_u^0}{d} \right\rfloor, \left\lfloor \frac{B_u^0}{4} \right\rfloor, \left\lfloor \frac{C_u^0}{7} \right\rfloor \right\}. \tag{8}$$

An owner that is offline for the whole trial contributes $K_u = 0$. Summing yields $K = \sum_u K_u$ because owners cannot pool their budgets. Reservation and full-path serialization can reach K when owners remain continuously online or offline, admitted requests have valid evidence, and processing is allowed to drain. A failure after a deposit debit or an early stage charge can instead leave fewer resources for completed work. This bound concerns how many complete paths the budgets can fund; it says nothing about throughput or latency and does not guarantee liveness.

Resource costs and delays are configured separately. The edge, block, and local-commitment delays are 2, 5, and 10 abstract units. Assigning 1 ms to a unit gives 17 ms of configured delay, to which queueing, communication, predicate execution, and runtime overhead must be added. Section IV measures these effects. They cannot be inferred from the resource-capacity bound.

E. CPN Traceability and Evidence Boundary

The stored CPN has eight pages, 72 places, 21 transitions, and 141 arcs. These inventory counts include receipt structures

TABLE II
TRACEABILITY OF THE EVALUATED PATH. RUNTIME RESERVATION AND TERMINAL SETTLEMENT ARE IMPLEMENTATION MECHANISMS, NOT FEATURES CLAIMED FOR THE UNCHANGED CPN SPECIFICATION.

Specification anchor	Retained condition and runtime interpretation
Request creation and deposit	Wallet sufficiency and deposit $d = 15$; runtime requests have separate unique identifiers.
`P_CID2SID`	Keys 0–63 map to four workers. Reusing a routing key does not reuse a request identifier.
`P_MobileRes` and resource guards	Costs $(2, 3)$, $(1, 2)$, $(1, 2)$ and non-offline stage guards are retained; full-path reservations are added in the gateway.
Candidate and proof stages	Selected evidence rules are transcribed. Candidate construction in the gateway and worker scheduling are service-allocation choices.
`VerifiedBC`, `END`	Commitment requires a candidate and a matching positive decision. A failed CPN guard does not produce a negative decision token; blocked termination is a runtime policy.
Commitment and timing	The configured delay becomes a local timer. Committee consensus and receipt-finality processing are outside the evaluated path.

that the evaluated path does not use; they are not reachable-state counts. A hash-pinned model file provides the cost constants, policy parameters, and 64-entry routing map. The service code implements selected guards and updates using those inputs without calling the CPN simulator. Table II distinguishes the specification elements from choices made for their runtime implementation.

The failure path illustrates why this distinction matters. In the CPN, a false guard disables a binding. The runtime instead treats a handled failed check as a blocked terminal outcome and releases the unused hold. This fail-fast rule and the chosen worker schedule retain selected conditions from the model, not all of its possible executions. Replay can check the accounting and ordering in recorded traces; it does not establish behavioral equivalence with the CPN or explore its complete reachable state space.

The evidence boundary is visible in the root predicates. They inspect the auxiliary deposit record at the head of the proof's transaction list, checking amount, destination, the sender/owner relation, transaction identity, and a deterministic proof-binding expression. A target parameter and a block-identifier threshold are checked as well. The evaluated threshold is zero, not a measured confirmation depth. The checks cover only the specified fields; they do not authenticate the full routed payload. Nor do their string-level signature dependencies perform public-key verification. Acceptance in this experiment means that those predicates succeeded. Authenticated requests, replay protection, concrete cryptographic checks, and a specified consensus/commit protocol would be needed for blockchain integration. The following evaluation therefore concerns accounted completion capacity and relative runtime under the four admission policies.

IV. RESULTS AND DISCUSSION

Section III gives the conditions under which an admitted request retains its remaining allowance. The experiments ask whether the implementation attains the associated completion bound and whether retaining concurrency is useful compared with per-participant serialization. We vary concurrency and arrival patterns to examine their effect on completion and runtime, then change modeled connectivity to test the limit of the accounting guarantee. As before, a measured commitment is a local record, not blockchain consensus finality.

A. Deployment and Experimental Design

The services ran under Python 3.13.5 on a Linux x86_64 host. The container exposed five logical processors with a four-CPU-equivalent quota and a 4 GiB memory limit. A driver used loopback TCP/JSON to communicate with six separate service processes: the gateway, four shard workers, and the root service. Topology, routing, stage costs, delays, and event logging were held constant across policies. With an abstract unit assigned 1 ms, edge packaging took a configured 2 ms, block generation 5 ms, and local commitment 10 ms. These are experimental settings, not a calibration against a wireless network or operational blockchain. Gateway code performs each account read, check, and update without an intervening asynchronous suspension.

A trial offered 256 distinct requests, split into 16 requests for each of 16 owners. Requests reused routing keys from the 64-key map, not their unique identifiers. We reset logical state between trials while keeping the service processes running. Twenty seeds selected workload order, mixed-profile owner assignments, and fault targets where applicable. Within each seed, configuration order was randomized. Matched policy runs have identical recorded input hashes, but sharing inputs does not make the runtime schedule deterministic. The retained data exclude four warm-up runs from the main campaign and a separate 44-trial pilot, leaving 880 trials and 225,280 submissions. The artifact contains their request responses and gateway events, together with environment information and source hashes.

The policy definitions are those in Section III. *Stagewise* checks the allowance at each charged stage. *Precheck* also tests the full path cost before dispatch, but records no hold. *Reservation* maintains the holds in Eqs. (2)–(6). *Serial* acquires a per-owner lock before applying the full-path precheck and retains it until the gateway receives the terminal response, including the local commitment delay. An occupied lock causes later requests from that owner to wait, not to be denied for being busy. Requests from other owners can proceed concurrently.

The main experiment combines the four profiles in Table III with four policies and 20 repetitions, yielding 320 trials. The default arrival pattern is a burst, with at most 16 outstanding service calls per owner at the driver. The computation-limited concurrency study adds caps of 1, 2, 4, and 8 for a further 320 trials; the cap-16 observations come from the main experiment. Staggered-arrival and full-scan studies each add 80 computation-limited trials, and later disconnection accounts for the final

TABLE III
INITIAL LOGICAL RESOURCE PROFILES. BALANCES ARE PER OWNER; K IS THE SYSTEM-WIDE BOUND IN EQ. (8). EACH OWNER OFFERS 16 REQUESTS.

Profile	W_u^0	B_u^0	C_u^0	K	Connectivity
Deposit-limited	100	100	100	96	All online
Computation-limited	300	100	35	80	All online
Battery-budget-limited	300	20	100	80	All online
Mixed	Four owner groups			96	One group offline

Mixed: four owners in each group, with (W_u^0, B_u^0, C_u^0) equal to $(300, 100, 100)$, $(300, 100, 35)$, $(300, 20, 100)$, or $(300, 100, 100)$ with continuous offline status. Their per-owner bounds are 14, 5, 5, and 0, respectively, giving $K = 4(14 + 5 + 5) = 96$.

80 trials. All studies use exact, fixed costs. Only connectivity changes in the disconnection study, not the cost vector.

B. Metrics and Validation Controls

Every submission in the retained data has a terminal response: committed, blocked after admission, or denied before dispatch. Denote these counts by N_c, N_b, and N_d, respectively, and the number admitted by N_a. They satisfy

$$N = N_c + N_b + N_d, \qquad N_a = N_c + N_b.$$

Completion over submissions is N_c/N, whereas completion over admissions is N_c/N_a. The latter applies only for $N_a > 0$ and excludes denied requests rather than treating them as successes. For static profiles, we also compare N_c with K. Attaining that resource bound need not make all 256 offered requests feasible.

Trial elapsed time begins at workload start and ends when all terminal responses have reached the driver. Request latency begins when the driver actually releases that request to its concurrency limiter and ends at its terminal response. It includes waits at the limiter, the gateway's owner lock, and downstream services, but excludes scheduling delay between a nominal arrival and actual driver release. For each trial, we calculate the mean and linearly interpolated 95th percentile of successful-request latencies, then average those summaries in the timing table. The P95 column is therefore an average of trial percentiles, not a percentile computed from pooled requests. Policies may complete different subsets, so these success-conditioned distributions do not measure latency effects on matched individual requests.

We measure partial-work cost as the computation allowance charged to requests that subsequently become blocked:

$$U_C = \sum_{x:\,\mathrm{outcome}(x)=\mathrm{blocked}} C_x^{\mathrm{spent}}.$$

Charges are recorded before the associated stage delays finish. Accordingly, U_C counts configured accounting units, not CPU time or physical energy. No modeled validation cost is charged to denied requests, although processing their denials takes time within the trial.

In operational mode, each policy uses constant-work local account checks. Aggregate reservations are fully reconstructed only after the timed interval and by a separate replay checker. The diagnostic study moves a full request-ledger scan into the execution path after every successful admission, stage charge, and terminal settlement, for all policies. Both modes keep logging and local assertions enabled. Their timings thus remain measurements of an instrumented prototype rather than an uninstrumented production system.

Nineteen unit tests passed. Read-only replay by a separate checker covered 781,126 gateway events from all 880 trials. Checks included balances, admission eligibility, stage order, deposit debits, unused-reservation settlement, and non-overlap of an owner's requests under serialization. For the 77,929 recorded commitments, replay also checked matching positive decisions and the configured commitment delay. None of these checked properties was violated in the retained traces. This provides evidence about those implementation executions, not exhaustive CPN reachability or model–code equivalence.

The tables give trial means and, where indicated, sample standard deviations. Selected policy comparisons pair trials by workload seed and use 95% Student-t intervals, $\bar{\Delta} \pm t_{0.975,19} s_\Delta / \sqrt{20}$. Analysis is at the trial level; its 256 correlated requests are not independent replications. These individual intervals are unadjusted and assume independent trial differences with an adequate t approximation. They characterize repeatability on this host, rather than performance across deployments or statistical equivalence of the policies.

C. Completion Capacity and Partial-Work Loss

The deposit-limited profile checks a case in which reservation should add no completions. Each owner can pay six deposits, giving $K = 16\lfloor 100/15 \rfloor = 96$. All policies admit and complete 96 requests and deny 160 (Table IV). The wallet check has already limited the work before battery or computation budgets can constrain validation.

For computation-limited owners, the bound is $K = 16\lfloor 35/7 \rfloor = 80$. Stagewise and Precheck fall short, with mean completions of 72.40 and 72.35, respectively; Reservation and Serial complete 80 in every repetition. The paired Reservation-minus-Stagewise difference is 7.60 requests, with a 95% interval of $[6.83, 8.37]$, corresponding to a 10.5% increase over the Stagewise mean. The burst explains the weak result for Precheck: all 256 requests in Stagewise or Precheck pass admission before the first block-stage charge. Nothing has yet been spent or reserved when their balances are checked. Reservation instead admits 80 and denies 176 before validation. Serial reaches the same counts by waiting for each owner's current request to terminate before checking the next.

Figure 2 also shows what the completion counts exclude. Reservation completes 100% of admitted requests, but only $80/256 = 31.25\%$ of submissions; the corresponding Stagewise figure is 28.28%. The submission-level gain is 2.97 percentage points, not 100% completion of the offered workload. To explain the shortfall, Stagewise charges a mean of 46.45 computation units to blocked requests and leaves 6.75 units unused across owners. Its accounting reconciles as $7(72.40) + 46.45 + 6.75 = 560$: completed work, partial

TABLE IV
MAIN OPERATIONAL-MODE OUTCOMES PER 256-REQUEST TRIAL, OVER 20 REPETITIONS. COMMITTED COUNTS ARE MEAN ± SAMPLE STANDARD DEVIATION; OTHER ENTRIES ARE MEANS. U_C IS COMPUTATION-BUDGET UNITS CHARGED TO BLOCKED REQUESTS.

Profile	Policy	Admitted	Committed	Blocked	Denied	U_C
Deposit-limited	Stagewise	96	96.00 ± 0.00	0.00	160	0.00
	Precheck	96	96.00 ± 0.00	0.00	160	0.00
	Reservation	96	96.00 ± 0.00	0.00	160	0.00
	Serial	96	96.00 ± 0.00	0.00	160	0.00
Computation-limited	Stagewise	256	72.40 ± 1.64	183.60	0	46.45
	Precheck	256	72.35 ± 1.69	183.65	0	46.80
	Reservation	80	80.00 ± 0.00	0.00	176	0.00
	Serial	80	80.00 ± 0.00	0.00	176	0.00
Battery-budget-limited	Stagewise	256	72.45 ± 1.79	183.55	0	45.45
	Precheck	256	72.40 ± 1.93	183.60	0	46.00
	Reservation	80	80.00 ± 0.00	0.00	176	0.00
	Serial	80	80.00 ± 0.00	0.00	176	0.00
Mixed	Stagewise	256	90.75 ± 1.65	165.25	0	36.90
	Precheck	192	90.25 ± 2.00	101.75	64	40.35
	Reservation	96	96.00 ± 0.00	0.00	160	0.00
	Serial	96	96.00 ± 0.00	0.00	160	0.00

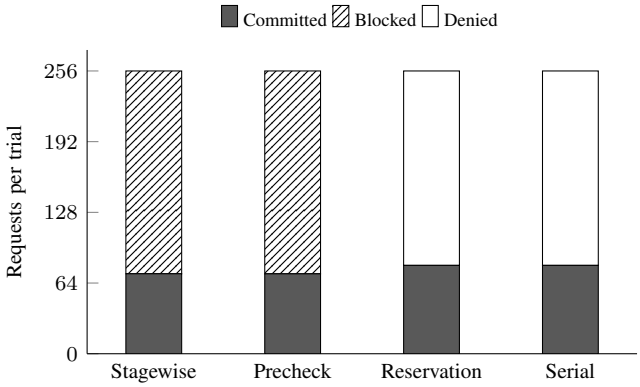


Fig. 2. Computation-limited mean outcomes. Each stacked bar accounts for 256 submissions; its committed and blocked portions together form the admitted count. Reservation and Serial both reach the 80-request resource bound.

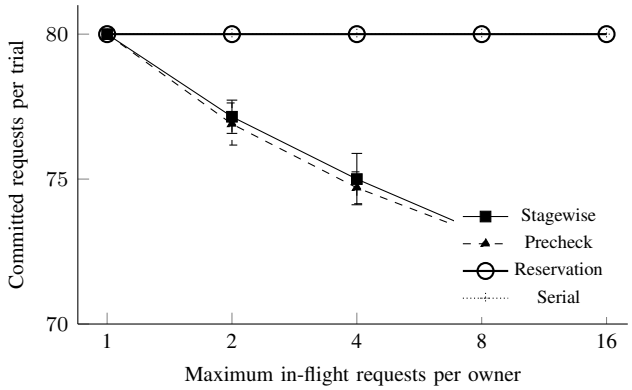


Fig. 3. Computation-limited mean completions against the per-owner driver cap. Error bars are 95% trial-mean intervals. Cap-16 observations are reused from the main experiment. Reservation and Serial coincide at 80; equal counts do not imply equal elapsed time.

spending, and residual balances account for the initial budget. Both the partial charges and the unusable residual allowance contribute to completing fewer than 80 requests. Reservation and Serial charge no computation allowance to blocked work.

The same effect appears when the logical battery budget is binding. Stagewise completes 72.45 requests on average; Reservation and Serial complete 80. The change of binding component is an accounting experiment, not a measurement of device energy. In the mixed profile, Reservation and Serial reach 96, while Stagewise completes 90.75 and Precheck 90.25. Precheck denies the 64 requests from initially offline owners, but it still allows online owners' requests to make overlapping claims on an unreserved balance. In each profile, the configuration determines the bound and the measurements show whether the admission policy reaches it.

D. Concurrency, Arrival Patterns, and Serialization

We next vary the number of outstanding driver calls per owner to examine the effect of overlap. The cap counts calls waiting for the Serial lock as well as calls being processed; it is not a bound on successful admissions. At cap 1, every policy completes 80 requests (Fig. 3). Stagewise still admits requests after usable allowance is exhausted, but these fail before the block-stage charge, giving $U_C = 0$. At caps 2, 4, 8, and 16, its mean completions decline to 77.15, 75.00, 73.15, and 72.40. Precheck follows the same general trend. Reservation and Serial remain at 80, although the increased driver cap does not change Serial's one-active- request restriction for each owner.

Spacing nominal arrivals 2 ms apart across the same 256-request stream raises mean completions to 75.70 for Stagewise and 77.10 for Precheck. Reservation and Serial still complete 80. More admission checks now occur after earlier spending becomes visible, which helps Precheck without making it

TABLE V
COMPUTATION-LIMITED TIMING IN MS. TRIAL ELAPSED TIMES ARE MEAN ± SAMPLE STANDARD DEVIATION. LATENCY COLUMNS AVERAGE PER-TRIAL SUCCESSFUL-REQUEST MEANS AND 95TH PERCENTILES IN OPERATIONAL MODE.

Policy	Operational	Full scan	Mean lat.	P95 lat.
Stagewise	234.4 ± 20.9	260.0 ± 20.1	126.9	187.8
Precheck	235.8 ± 20.3	255.4 ± 20.8	127.0	187.5
Reservation	207.8 ± 26.8	207.4 ± 19.3	118.9	183.8
Serial	205.8 ± 15.7	215.1 ± 22.9	112.1	184.9

TABLE VI
LATER-DISCONNECTION MEAN OUTCOMES PER 256-REQUEST TRIAL OVER 20 REPETITIONS. U_C IS COMPUTATION-BUDGET UNITS CHARGED TO BLOCKED REQUESTS.

Policy	Admitted	Committed	Blocked	Denied	U_C
Stagewise	256	163.00	93.00	0	55.25
Precheck	256	162.90	93.10	0	56.35
Reservation	224	168.00	56.00	32	14.70
Serial	172	168.00	4.00	84	12.00

equivalent to a reservation. The nominal span from the first arrival to the last is 510 ms and is included in trial elapsed time. Burst and staggered trial durations therefore reflect different offered-load patterns.

The capacity result must be read separately from timing. At cap 16 in the computation-limited profile, Reservation takes a mean of 207.8 ms per trial and Serial 205.8 ms. The paired Reservation-minus-Serial difference is 2.0 ms, with a 95% interval of $[-8.0, 12.1]$ ms. This interval does not establish a timing advantage or show equivalence. In this configuration, serialization reaches the bound without a demonstrated runtime penalty for forbidding overlap within an owner.

Mixed resources provide a clearer timing distinction at the same completion count. Both policies complete 96 requests, but mean trial times are 220.6 ms for Reservation and 377.2 ms for Serial. The paired Serial-minus-Reservation difference is 156.6 ms, with a 95% interval of $[144.9, 168.3]$ ms. An owner in the least constrained group can afford 14 requests. Under Reservation, these paths can overlap; under Serial, the next request waits until the terminal response releases the owner lock. This is consistent with the shorter trial time, although the measurements do not isolate individual scheduling components. The result describes how long the complete workload, including denials, takes to drain. It does not imply that every individual request has lower latency.

E. Runtime Instrumentation and Connectivity Loss

Table V separates operational accounting from the diagnostic cost of scanning the request ledger during execution. Full scans increase mean computation-limited trial time from 234.4 to 260.0 ms for Stagewise and from 235.8 to 255.4 ms for Precheck. For Reservation, the means are 207.8 and 207.4 ms, giving a paired full-minus-operational difference of -0.4 ms with a 95% interval of $[-14.0, 13.1]$ ms. That estimate cannot be taken as a speedup from extra checking. Scans add work to account updates and may alter the schedule; the aggregate timing does not separate these effects. We therefore compare policies using the common local-check mode and report full-scan timings as a separate diagnostic study. The other supplementary study changes connectivity after admission.

The later-disconnection trial begins with every owner online and with wallet, battery, and computation balances of 300, 100, and 100 units. The workload seed selects four owners. For each selected owner, the runtime sets its connectivity flag to offline when its first successfully charged block stage finishes the configured delay, before that request attempts a proof-stage charge. The services and loopback connections keep running, so they can return blocked responses and release unused reservations. The fault changes the connectivity condition in A5, not the transport connection, and does not test recovery from a service crash. Root commitment retains its configured timer and does not recheck connectivity after acceptance.

Table VI reports Reservation's 224 admissions, 168 commitments, 56 blocked requests, and 32 denials. Serial completes 168 from 172 admissions, with four blocked requests and 84 denials. Unaffected owners each support 14 requests, giving $12 \times 14 = 168$ commitments; affected owners stop before completion. Serial exposes one admitted request per affected owner, then denies subsequent requests at the connectivity precheck. Reservation has already admitted 14 per affected owner, leaving more unfinished work exposed to the change.

Only 75% of Reservation's admissions complete after the fault, although both policies complete $168/256 = 65.625\%$ of submissions. Blocked counts are not proportional to consumed allowance: U_C is 14.70 for Reservation and 12.00 for Serial, despite 56 versus four blocked requests. Many encounter the offline condition before any resource charge. Terminal handling releases unused holds, not spent resources or deposit debits. The invariant remains satisfied, but more admitted paths can no longer proceed.

F. Interpretation, Limitations, and Deployment Implications

These results distinguish admission limits from losses during execution. When the wallet pays for too few deposits, all policies stop admitting work before validation resources become scarce. When the validation allowance binds instead, concurrent requests can pass every stagewise check yet leave spending on paths that do not finish. Reservation avoids that overcommitment; per-participant serialization reaches the same capacity by waiting between requests. The remaining choice is whether the time saved by permitting overlap warrants the larger set of in-flight obligations. The mixed-profile timing result shows a benefit, and the connectivity experiment shows the associated exposure.

The workloads contain synthetic deposit and identity records, use fixed routing, and assign the same stage-specific costs to every request. Costs differ by stage, not by request or trial. The experiments do not cover contract-state conflicts, variable bytecode paths, heterogeneous transaction costs, or estimation error. Overestimating a cost could deny otherwise

feasible work; underestimating it would invalidate the fixed-cost feasibility argument. In addition, all duties charge a single logical owner. Handling separately owned device, shard, and root budgets would require a multi-account mechanism and its own evaluation.

Centralized accounting and candidate-state management may make the gateway a bottleneck. Atomic event-loop updates do not provide replication or crash tolerance. The reservation mechanism has neither leases nor recovery, and settlement depends on reaching handled termination. A protocol that released an apparently abandoned hold would also have to prevent delayed work from spending the allowance after release. Changing the modeled offline flag does not test that situation. Similarly, the valid symbolic records used in the performance workloads establish neither payload authenticity nor Byzantine agreement or cross-shard atomicity.

The single-host deployment excludes wide-area communication, mobile radio behavior, measured energy, and independent-host scaling. Logical resets and randomized configuration order cannot rule out host-state carryover from reused processes. Timing remains instrumented and workload-dependent; intervals over 20 repetitions do not establish production performance. Replay checks observed accounting and ordering, not all possible executions. The evidence supports completion-bound attainment under the successful-path assumptions and useful concurrency in some workloads. It establishes neither production blockchain throughput nor completion of every admitted request.

V. Conclusion

A non-negative balance is not enough to protect completion under concurrent validation. Requests may spend their available allowance on early stages and leave too little for the rest of the path. MobiShard records the remaining cost as a reservation before shard dispatch, keeping the charged account separate from the execution host. Under fixed costs and the specified atomic updates, the invariant prevents one admitted request from using another's reserved allowance. That accounting protection operates alongside, rather than in place of, evidence and connectivity checks.

Across 880 trials comparing four policies, Reservation and per-participant serialization both reached the configured capacity bound in the tested static profiles without resource-related partial-work loss. In computation-limited bursts, they each completed 80 of 256 submissions, compared with a Stagewise mean of 72.40. Equal completion counts leave concurrency, not additional capacity, as the reason to choose reservation. It reduced mean trial elapsed time in the mixed profile, while the main computation-limited comparison showed no clear timing advantage. It also exposed more admitted requests to interruption when modeled connectivity changed. The evaluation thus identifies both a use for overlapping validation and a cost to accepting more work in advance.

The findings concern logical budgets and local commitment in a trusted, single-host prototype. Further work will address variable cost estimates, separately owned budgets, and reservation leases with recovery rules that prevent delayed work from spending released allowance. Concrete cryptographic verification and consensus are needed before blockchain-level guarantees can be evaluated. Reservation preserves budget for concurrent validation; completion still depends on valid evidence, connectivity at guarded stages, and responsive services.

References

[1] M. J. Amiri, Z. Lai, L. Patel, B. T. Loo, E. Lo, and W. Zhou, "Saguaro: An edge computing-enabled hierarchical permissioned blockchain," in *Proc. IEEE Int. Conf. Data Engineering (ICDE)*, 2023, pp. 259–272.

[2] M. J. Amiri, D. Agrawal, and A. El Abbadi, "SharPer: Sharding permissioned blockchains over network clusters," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2021.

[3] J. Hellings and M. Sadoghi, "ByShard: Sharding in a byzantine environment," *Proc. VLDB Endowment*, vol. 14, no. 11, pp. 2230–2243, 2021.

[4] S. Das, V. Krishnan, and L. Ren, "Efficient cross-shard transaction execution in sharded blockchains," arXiv preprint arXiv:2007.14521v2, 2021. [Online]. Available: https://arxiv.org/abs/2007.14521v2

[5] Y. Zhang, S. Pan, and J. Yu, "TxAllo: Dynamic transaction allocation in sharded blockchain systems," in *Proc. IEEE Int. Conf. Data Engineering (ICDE)*, 2023, pp. 721–733.

[6] H. Huang, X. Peng, J. Zhan, S. Zhang, Y. Lin, Z. Zheng, and S. Guo, "BrokerChain: A cross-shard blockchain protocol for account/balance-based state sharding," in *Proc. IEEE INFOCOM*, 2022, pp. 1968–1977.

[7] D. Crankshaw, G.-E. Sela, C. Zumar, X. Mo, J. E. Gonzalez, I. Stoica, and A. Tumanov, "InferLine: Latency-aware provisioning and scaling for prediction serving pipelines," in *Proc. ACM Symp. Cloud Computing (SoCC)*, 2020, pp. 477–491.

[8] P. Li, M. Song, M. Xing, Z. Xiao, Q. Ding, S. Guan, and J. Long, "SPRING: Improving the throughput of sharding blockchain via deep reinforcement learning based state placement," in *Proc. ACM Web Conf.*, 2024, pp. 2836–2846.

[9] Z. Hong, S. Guo, E. Zhou, W. Chen, H. Huang, and A. Zomaya, "GriDB: Scaling blockchain database via sharding and off-chain cross-shard mechanism," arXiv preprint arXiv:2407.03750, 2024. [Online]. Available: https://arxiv.org/abs/2407.03750

[10] C. Huang, Z. Wang, H. Chen, Q. Hu, Q. Zhang, W. Wang, and X. Guan, "RepChain: A reputation-based secure, fast, and high incentive blockchain system via sharding," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4291–4304, 2021.

[11] X. Huang, W. Jie, S. Zhang, H. Yang, W. Qiu, Q. Zhang, H. Huang, Z. Xiong, S. Tang, H. Zheng, and Z. Zheng, "ContribChain: A stress-balanced blockchain sharding protocol with node contribution awareness," arXiv preprint arXiv:2505.06899, 2025. [Online]. Available: https://arxiv.org/abs/2505.06899

[12] M. Li, Y. Lin, and J. Zhang, "StableShard: Stable and scalable blockchain sharding with high concurrency via collaborative committees," arXiv preprint arXiv:2407.06882v2, 2025. [Online]. Available: https://arxiv.org/abs/2407.06882v2

[13] Y. Lin, M. Li, and J. Zhang, "SpiralShard: Highly concurrent and secure blockchain sharding via linked cross-shard endorsement," *IEEE/ACM Trans. Netw.*, 2025.

[14] Z. Hong, S. Guo, E. Zhou, J. Zhang, W. Chen, J. Liang, J. Zhang, and A. Zomaya, "Prophet: Conflict-free sharding blockchain via byzantine-tolerant deterministic ordering," arXiv preprint arXiv:2304.08595, 2023. [Online]. Available: https://arxiv.org/abs/2304.08595

[15] R. Adhikari and C. Busch, "Lockless blockchain sharding with multiversion control," arXiv preprint arXiv:2303.17105, 2023. [Online]. Available: https://arxiv.org/abs/2303.17105

[16] R. Gelashvili, A. Spiegelman, Z. Xiang, G. Danezis, Z. Li, D. Malkhi, Y. Xia, and R. Zhou, "Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing," in *Proc. ACM PPoPP*, 2023.

[17] L. Qin, H. Lu, Y. Chen, Z. Gu, D. Zhao, and F. Wu, "Energy-efficient blockchain-enabled user-centric mobile edge computing," *IEEE Transactions on Cognitive Communications and Networking*, vol. 10, no. 4, pp. 1452–1466, 2024.

[18] Z. Wang, Q. Hu, Z. Xiong, Y. Liu, and D. Niyato, "Resource optimization for blockchain-based federated learning in mobile edge computing," *IEEE Internet of Things Journal*, vol. 11, no. 9, pp. 15 166–15 178, 2024.

[19] Z. Zhang, G. Yu, C. Sun, X. Wang, Y. Wang, M. Zhang, W. Ni, R. P. Liu, A. Reeves, and N. Georgalas, "TBDD: A new trust-based, DRL-driven framework for blockchain sharding in IoT," *Computer Networks*, vol. 244, p. 110343, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389128624001750

[20] B. Bünz, L. Kiffer, L. Luu, and M. Zamani, "FlyClient: Super-light clients for cryptocurrencies," in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 928–946.

[21] P. Vesely, K. Gurkan, M. Straka, A. Gabizon, P. Jovanovic, G. Konstantopoulos, A. Oines, M. Olszewski, and E. Tromer, "Plumo: An ultralight blockchain client," in *Financial Cryptography and Data Security*, 2022.

[22] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like Clockwork: Performance predictability from the bottom up," in *Proc. USENIX OSDI*, 2020, pp. 443–462.

[23] Y. Zhang, W. Hua, Z. Zhou, G. E. Suh, and C. Delimitrou, "Sinan: Data-driven, QoS-aware cluster management for microservices," in *Proc. ACM ASPLOS*, 2021.

[24] J. Fried, Z. Ruan, A. Ousterhout, and A. Belay, "Caladan: Mitigating interference at microsecond timescales," in *Proc. USENIX OSDI*, 2020, pp. 281–297.

[25] T. Diamandis, A. Evans, T. Chitra, and G. Angeris, "Dynamic pricing for non-fungible resources: Designing multidimensional blockchain fee markets," arXiv preprint arXiv:2208.07919, 2022. [Online]. Available: https://arxiv.org/abs/2208.07919

[26] G. Angeris, T. Diamandis, and C. Moallemi, "Multidimensional blockchain fees are (essentially) optimal," arXiv preprint arXiv:2402.08661, 2024. [Online]. Available: https://arxiv.org/abs/2402.08661

[27] N. Lavee, N. Nisan, M. Pai, and M. Resnick, "Does your blockchain need multidimensional transaction fees?" arXiv preprint arXiv:2504.15438, 2025. [Online]. Available: https://arxiv.org/abs/2504.15438

[28] E. Verbeek and D. Fahland, "CPN IDE: An extensible replacement for CPN Tools that uses Access/CPN," in *Proc. Int. Conf. Process Mining (ICPM), Demonstrations*, 2021, pp. 29–30.

[29] R. M. Dijkman, "SimPN: A Python library for modeling and simulating timed, colored Petri nets," in *Proc. BPM Demonstrations and Resources Forum*, ser. CEUR Workshop Proceedings, vol. 3758, 2024, pp. 71–75.

[30] A. Berti and W. M. P. van der Aalst, "CPN-Py: A Python-based tool for modeling and analyzing colored Petri nets," arXiv preprint arXiv:2506.12238, 2025. [Online]. Available: https://arxiv.org/abs/2506.12238